

Lab0实验报告

思考题

Thinking 0.1

Thinking 0.1 思考下列有关 Git 的问题：

- 在前述已初始化的 `~/learnGit` 目录下，创建一个名为 `README.txt` 的文件。执行命令 `git status > Untracked.txt`（其中的 `>` 为输出重定向，我们将在 0.6.3 中详细介绍）。
- 在 `README.txt` 文件中添加任意文件内容，然后使用 `add` 命令，再执行命令 `git status > Stage.txt`。
- 提交 `README.txt`，并在提交说明里写入自己的学号。
- 执行命令 `cat Untracked.txt` 和 `cat Stage.txt`，对比两次运行的结果，体会 `README.txt` 两次所处位置的不同。
- 修改 `README.txt` 文件，再执行命令 `git status > Modified.txt`。
- 执行命令 `cat Modified.txt`，观察其结果和第一次执行 `add` 命令之前的 `status` 是否一样，并思考原因。

不同，第一次执行 `add` 之前的 `status` 中没有 `Stage.txt` 和 `Modified.txt`，所以 `Untracked files` 也没有。第一次 `add` 之前，`README.txt` 在工作区，且不在版本库中，属于 `Untracked`；而第二次时 `README.txt` 在版本库中已经有一个版本，但是工作区做了修改，即 `Modified`。

Thinking 0.2

Thinking 0.2 仔细看看 0.10，思考一下箭头中的 `add the file`、`stage the file` 和 `commit` 分别对应的是 Git 里的哪些命令呢？

前两者都为 `git add`，第三者为 `git commit`

Thinking 0.3

Thinking 0.3 思考下列问题：

1. 代码文件 `print.c` 被错误删除时，应当使用什么命令将其恢复？
 2. 代码文件 `print.c` 被错误删除后，执行了 `git rm print.c` 命令，此时应当使用什么命令将其恢复？
 3. 无关文件 `hello.txt` 已经被添加到暂存区时，如何在不删除此文件的前提下将其移出暂存区？
1. `git checkout -- printf.c`
 2. `git checkout HEAD^ -- print.c`
 3. `git reset HEAD -- hello.txt`

Thinking 0.4

Thinking 0.4 思考下列有关 Git 的问题：

- 找到在 `/home/22xxxxxx/learnGit` 下刚刚创建的 `README.txt` 文件，若不存在则新建该文件。
- 在文件里加入 `Testing 1`，`git add`，`git commit`，提交说明记为 `1`。

- 模仿上述做法，把 1 分别改为 2 和 3，再提交两次。
- 使用 `git log` 命令查看提交日志，看是否已经有三次提交，记下提交说明为 3 的哈希值 a 。
- 进行版本回退。执行命令 `git reset --hard HEAD^` 后，再执行 `git log`，观察其变化。
- 找到提交说明为 1 的哈希值，执行命令 `git reset --hard` 后，再执行 `git log`，观察其变化。
- 现在已经回到了旧版本，为了再次回到新版本，执行 `git reset --hard`，再执行 `git log`，观察其变化。

提交三次后会有三次提交。

执行 `git reset --hard HEAD^` 后回退一个版本，只能看到“1”和“2”，看不到第三次了。

回退到第一次提交的版本后，看不到后两次了。

执行 `git reset --hard <第三次的哈希值>` 后，又能看到三次提交记录了。

Thinking 0.5

Thinking 0.5 执行如下命令，并查看结果

- `echo first`
- `echo second > output.txt`
- `echo third > output.txt`
- `echo forth >> output.txt`

1. 终端输出 `first`
2. 把 `second` 写入 `output.txt`
3. 把 `third` 写入 `output.txt`，覆盖上一次写入的内容
4. 把 `forth` 追加写入到 `output.txt`，不覆盖之前的内容

Thinking 0.6

Thinking 0.6 使用你知道的方法（包括重定向）创建下图内容的文件（文件命名为 `test`），将创建该文件的命令序列保存在 `command` 文件中，并将 `test` 文件作为批处理文件运行，将运行结果输出至 `result` 文件中。给出 `command` 文件和 `result` 文件的内容，并对最后的结果进行解释说明（可以从 `test` 文件的内容入手）。具体实现的过程中思考下列问题：`echo echo shell start` 与 `echo 'echo shell start'` 效果是否有区别；`echo echo $c>file1` 与 `echo 'echo $c>file1'` 效果是否有区别。（前面的单引号改成反引号）

有区别：反引号会实现里面的命令，将其标准输出作为字符串替换，加上反引号后，里面的命令会被执行，执行得到的标准输出再被外层的 `echo` 输出。

`command` 文件内容：

```
1 #!/bin/bash
2 echo 'echo shell start...' > test
3 echo 'echo set a = 1' >> test
4 echo 'a=1' >> test
5 echo 'echo set b = 2' >> test
6 echo 'b=2' >> test
7 echo 'echo set c = a+b' >> test
8 echo 'c=[$a+$b]' >> test
9 echo 'echo c = $c' >> test
10 echo 'echo save c to ./file1' >> test
```

```

11 echo 'echo $c>file1' >> test
12 echo 'echo save b to ./file2' >> test
13 echo 'echo $b>file2' >> test
14 echo 'echo save a to ./file3' >> test
15 echo 'echo $a>file3' >> test
16 echo 'echo save file1 file2 file3 to file4' >> test
17 echo 'cat file1>file4' >> test
18 echo 'cat file2>>file4' >> test
19 echo 'cat file3>>file4' >> test
20 echo 'echo save file4 to ./result' >> test
21 echo 'cat file4>>./result' >> test

```

result文件内容:

```

1 3
2 2
3 1

```

运行test的输出

```

1 Shell Start...
2 set a = 1
3 set b = 2
4 set c = a+b
5 c = 3
6 save c to ./file1
7 save b to ./file2
8 save a to ./file3
9 save file1 file2 file3 to file4
10 save file4 to ./result

```

test文件内容:

```

1 echo Shell Start...
2 echo set a = 1
3 a=1
4 echo set b = 2
5 b=2
6 echo set c = a+b
7 c=${a+$b}
8 echo c = $c
9 echo save c to ./file1
10 echo $c>file1
11 echo save b to ./file2
12 echo $b>file2
13 echo save a to ./file3
14 echo $a>file3
15 echo save file1 file2 file3 to file4
16 cat file1>file4
17 cat file2>>file4
18 cat file3>>file4
19 echo save file4 to ./result
20 cat file4>>./result

```

解释说明，test文件中的非指令声明的变量a、b、c，然后把c、b、a输入到了file1、2、3中，最后再把file1、2、3输出到file4，之后把file4输出到result。因此有输出3、2、1，对应c、b、a的值。

线上练习

Exercise 0.1

没什么难点，最基本的 gcc 调用和 Makefile 编写，以及 cp 命令

Exercise 0.2

```
1  #!/bin/bash
2  a=1
3  while [ $a -le 100 ]
4  do
5      if [ $a -gt 70 ]          #if loop variable is greater than 70
6      then
7          # 删除目录
8          rm -r "file$a"
9
10         elif [ $a -gt 40 ]      # else if loop variable is great than 40
11         then
12             #重命名目录
13             mv "file$a" "newfile$a"
14         fi
15             #don't forget change the loop variable
16         a=$((a+1))
17     done
```

需要注意条件语句的语法规则，不能忽略必须的空格

Exercise 0.3

```
1  #!/bin/bash
2  #First you can use grep (-n) to find the number of lines of string.
3  #Then you can use awk to separate the answer.
4  file="$1"
5  int="$2"
6  result="$3"
7  grep -nwo "$int" "$file" | cut -d: -f1 > "$result"
```

调用脚本时传入参数的运用，难点在于：

- grep 的参数，-nwo 表示显示行号、匹配整个单词、只输出匹配部分
- 管道符使用
- cut 指令使用，指出分隔符为：以及输出第一个字段

Exercise 0.4

分为几个部分

第一部分是 sed 的使用

```
1 #!/bin/bash
2 filename="$1"
3 old="$2"
4 new="$3"
5 sed -i "s/$old/$new/g" "$filename"
```

第二部分刚开始时难以理解，后面查询资料后得知要用外层的 `Makefile` 调用内层的 `Makefile`

```
1 # 外层
2 all: fibo
3 fibo: code/fibo.o code/main.o
4     gcc $^ -o $@
5 code/fibo.o code/main.o:
6     $(MAKE) -C code
7 clean:
8     $(MAKE) -C code clean
```

这里需要注意，`$^` 表示依赖项，`$@` 表示标签，这里为 `fibo`。

调用 `make` 时要写 `$(MAKE)`

```
1 # 内层
2 all: fibo.o main.o
3 %.o: %.c
4     gcc -I../include -c $< -o $@
5 clean:
6     rm -f fibo.o main.o
```

注意用 `-I` 来指定库函数目录

实验体会

总的来说第一次实验主要是对基础操作的熟悉。注意到有许多的命令都有着十分复杂的参数，需要我多加记忆了解与练习。

此外，`Makefile`与`bash`的脚本编写也对我有着挑战性，其中的语法需要我多加记忆。

比较麻烦的是，遇到需要编写C语言与MIPS汇编语言的部分时，我发现自己对它们的知识已经忘记许多了，需要我重新熟悉起来。